



CPSC 100

Computational Thinking

Data Representation

Instructor: Firas Moosvi
Department of Computer Science
University of British Columbia



Agenda

- Course Admin
- Learning Goals
- Intro to Data Representation

Course Admin



Course Admin

- Test 2 released back to you!
- Test 3 released back to you!
- I changed the labs so that you can still view it after you submit them - sorry about that, this was always the intention!
- Lab grading underway...
- Go to your labs this week! Project groups will be made by Friday!



Learning Goals

After this **today's lecture**, you should be able to:

- Understand the usage of data before and after computers.
- Recognize **hexadecimal numbers** and their role in data representation.
- Count in different **standard number bases** (i.e. 2, 10, 16).
- **Translate** numbers between binary, hexadecimal, and decimal without a calculator

After watching the **take home-video**, you should be able to:

- Recognize the difference between binary and ternary numbers
- Translate numbers between binary, hexadecimal, decimal and **ternary**



Q: What will the following expression report?



iClicker

- A. True
- B. False





Q: What will the following expression report?



iClicker

A. True

B. False



Data Representation



Data Representation Before Computers



Digital Data Before Computers

Creative computational thinkers have invented digital data representation systems for millennia, adapting to many different technologies:

Numeral systems

0123456789

·|ʹʹʹ07VΛ9

I II III IV V VI VII VIII IX X

o 1 2 3 4 5 6 7 8 9 10

o 1 2 3 4 5 6 7 8 9 10

o 1 2 3 4 5 6 7 8 9 10

○一 二 三 四 五 六 七 八 九

Hindu–Arabic numeral system

hənqəminəm alphabet 1

c celəx hand	č čəʔt put it on top of	č mink čəčiʔqən	h hiləm fall off, roll	k wait! ket čxʷ ʔaʔ
kʷ kʷasən star	kʷ kʷəwyəkʷ fish hook	l leləm house	lʰ spa:l Raven	š šəʔəm salt
t knife təctən	m mənə child	mʰ nem go	n netəʔ early morning	nʰ tin hummingbird
p pipá:m frog	pʰ patʰəs cradleboard	q qeqən housepost	qʰ qəwət drum	qʷ qʷənəs whale
qʷ qʷi:n ear	s wolf stqayəʔ	š purse šxʷteləʔelə	t ten mother	tʰ tʰələm singing
tʰ bone stʰam	θ tree θqet	w throw it welx	wʰ playing hiwáfəm	x sea lion xes

© 1999/2003 Musqueam Indian Band & UBC FNELG Musqueam Language Program.

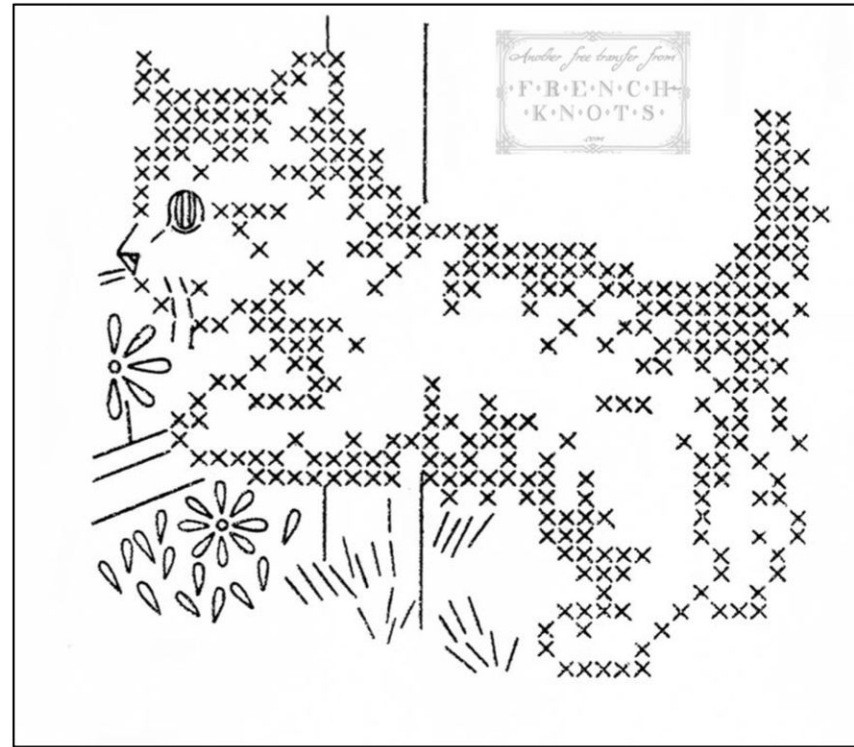
Illustrated by: jshil, Ann Campbell.

Phoenician alphabet

𐤀	ʾ	𐤁	𐤂	𐤃	𐤄	𐤅	𐤆	𐤇	𐤈	𐤉	𐤊	𐤋	𐤌	𐤍	𐤎	𐤏	𐤐	𐤑	𐤒	𐤓	𐤔	𐤕	𐤖	𐤗	𐤘	𐤙	𐤚	𐤛	𐤜	𐤝	𐤞	𐤟	𐤠	𐤡	𐤢	𐤣	𐤤	𐤥	𐤦	𐤧	𐤨	𐤩	𐤪	𐤫	𐤬	𐤭	𐤮	𐤯	𐤰	𐤱	𐤲	𐤳	𐤴	𐤵	𐤶	𐤷	𐤸	𐤹	𐤺	𐤻	𐤼	𐤽	𐤾	𐤿	𐥀	𐥁	𐥂	𐥃	𐥄	𐥅	𐥆	𐥇	𐥈	𐥉	𐥊	𐥋	𐥌	𐥍	𐥎	𐥏	𐥐	𐥑	𐥒	𐥓	𐥔	𐥕	𐥖	𐥗	𐥘	𐥙	𐥚	𐥛	𐥜	𐥝	𐥞	𐥟	𐥠	𐥡	𐥢	𐥣	𐥤	𐥥	𐥦	𐥧	𐥨	𐥩	𐥪	𐥫	𐥬	𐥭	𐥮	𐥯	𐥰	𐥱	𐥲	𐥳	𐥴	𐥵	𐥶	𐥷	𐥸	𐥹	𐥺	𐥻	𐥼	𐥽	𐥾	𐥿	𐦀	𐦁	𐦂	𐦃	𐦄	𐦅	𐦆	𐦇	𐦈	𐦉	𐦊	𐦋	𐦌	𐦍	𐦎	𐦏	𐦐	𐦑	𐦒	𐦓	𐦔	𐦕	𐦖	𐦗	𐦘	𐦙	𐦚	𐦛	𐦜	𐦝	𐦞	𐦟	𐦠	𐦡	𐦢	𐦣	𐦤	𐦥	𐦦	𐦧	𐦨	𐦩	𐦪	𐦫	𐦬	𐦭	𐦮	𐦯	𐦰	𐦱	𐦲	𐦳	𐦴	𐦵	𐦶	𐦷	𐦸	𐦹	𐦺	𐦻	𐦼	𐦽	𐦾	𐦿	𐧀	𐧁	𐧂	𐧃	𐧄	𐧅	𐧆	𐧇	𐧈	𐧉	𐧊	𐧋	𐧌	𐧍	𐧎	𐧏	𐧐	𐧑	𐧒	𐧓	𐧔	𐧕	𐧖	𐧗	𐧘	𐧙	𐧚	𐧛	𐧜	𐧝	𐧞	𐧟	𐧠	𐧡	𐧢	𐧣	𐧤	𐧥	𐧦	𐧧	𐧨	𐧩	𐧪	𐧫	𐧬	𐧭	𐧮	𐧯	𐧰	𐧱	𐧲	𐧳	𐧴	𐧵	𐧶	𐧷	𐧸	𐧹	𐧺	𐧻	𐧼	𐧽	𐧾	𐧿	𐨀	𐨁	𐨂	𐨃	𐨄	𐨅	𐨆	𐨇	𐨈	𐨉	𐨊	𐨋	𐨌	𐨍	𐨎	𐨏	𐨐	𐨑	𐨒	𐨓	𐨔	𐨕	𐨖	𐨗	𐨘	𐨙	𐨚	𐨛	𐨜	𐨝	𐨞	𐨟	𐨠	𐨡	𐨢	𐨣	𐨤	𐨥	𐨦	𐨧	𐨨	𐨩	𐨪	𐨫	𐨬	𐨭	𐨮	𐨯	𐨰	𐨱	𐨲	𐨳	𐨴	𐨵	𐨶	𐨷	𐨸	𐨹	𐨺	𐨻	𐨼	𐨽	𐨾	𐨿	𐩀	𐩁	𐩂	𐩃	𐩄	𐩅	𐩆	𐩇	𐩈	𐩉	𐩊	𐩋	𐩌	𐩍	𐩎	𐩏	𐩐	𐩑	𐩒	𐩓	𐩔	𐩕	𐩖	𐩗	𐩘	𐩙	𐩚	𐩛	𐩜	𐩝	𐩞	𐩟	𐩠	𐩡	𐩢	𐩣	𐩤	𐩥	𐩦	𐩧	𐩨	𐩩	𐩪	𐩫	𐩬	𐩭	𐩮	𐩯	𐩰	𐩱	𐩲	𐩳	𐩴	𐩵	𐩶	𐩷	𐩸	𐩹	𐩺	𐩻	𐩼	𐩽	𐩾	𐩿	𐪀	𐪁	𐪂	𐪃	𐪄	𐪅	𐪆	𐪇	𐪈	𐪉	𐪊	𐪋	𐪌	𐪍	𐪎	𐪏	𐪐	𐪑	𐪒	𐪓	𐪔	𐪕	𐪖	𐪗	𐪘	𐪙	𐪚	𐪛	𐪜	𐪝	𐪞	𐪟	𐪠	𐪡	𐪢	𐪣	𐪤	𐪥	𐪦	𐪧	𐪨	𐪩	𐪪	𐪫	𐪬	𐪭	𐪮	𐪯	𐪰	𐪱	𐪲	𐪳	𐪴	𐪵	𐪶	𐪷	𐪸	𐪹	𐪺	𐪻	𐪼	𐪽	𐪾	𐪿	𐫀	𐫁	𐫂	𐫃	𐫄	𐫅	𐫆	𐫇	𐫈	𐫉	𐫊	𐫋	𐫌	𐫍	𐫎	𐫏	𐫐	𐫑	𐫒	𐫓	𐫔	𐫕	𐫖	𐫗	𐫘	𐫙	𐫚	𐫛	𐫜	𐫝	𐫞	𐫟	𐫠	𐫡	𐫢	𐫣	𐫤	𐫥	𐫦	𐫧	𐫨	𐫩	𐫪	𐫫	𐫬	𐫭	𐫮	𐫯	𐫰	𐫱	𐫲	𐫳	𐫴	𐫵	𐫶	𐫷	𐫸	𐫹	𐫺	𐫻	𐫼	𐫽	𐫾	𐫿	𐬀	𐬁	𐬂	𐬃	𐬄	𐬅	𐬆	𐬇	𐬈	𐬉	𐬊	𐬋	𐬌	𐬍	𐬎	𐬏	𐬐	𐬑	𐬒	𐬓	𐬔	𐬕	𐬖	𐬗	𐬘	𐬙	𐬚	𐬛	𐬜	𐬝	𐬞	𐬟	𐬠	𐬡	𐬢	𐬣	𐬤	𐬥	𐬦	𐬧	𐬨	𐬩	𐬪	𐬫	𐬬	𐬭	𐬮	𐬯	𐬰	𐬱	𐬲	𐬳	𐬴	𐬵	𐬶	𐬷	𐬸	𐬹	𐬺	𐬻	𐬼	𐬽	𐬾	𐬿	𐭀	𐭁	𐭂	𐭃	𐭄	𐭅	𐭆	𐭇	𐭈	𐭉	𐭊	𐭋	𐭌	𐭍	𐭎	𐭏	𐭐	𐭑	𐭒	𐭓	𐭔	𐭕	𐭖	𐭗	𐭘	𐭙	𐭚	𐭛	𐭜	𐭝	𐭞	𐭟	𐭠	𐭡	𐭢	𐭣	𐭤	𐭥	𐭦	𐭧	𐭨	𐭩	𐭪	𐭫	𐭬	𐭭	𐭮	𐭯	𐭰	𐭱	𐭲	𐭳	𐭴	𐭵	𐭶	𐭷	𐭸	𐭹	𐭺	𐭻	𐭼	𐭽	𐭾	𐭿	𐮀	𐮁	𐮂	𐮃	𐮄	𐮅	𐮆	𐮇	𐮈	𐮉	𐮊	𐮋	𐮌	𐮍	𐮎	𐮏	𐮐	𐮑	𐮒	𐮓	𐮔	𐮕	𐮖	𐮗	𐮘	𐮙	𐮚	𐮛	𐮜	𐮝	𐮞	𐮟	𐮠	𐮡	𐮢	𐮣	𐮤	𐮥	𐮦	𐮧	𐮨	𐮩	𐮪	𐮫	𐮬	𐮭	𐮮	𐮯	𐮰	𐮱	𐮲	𐮳	𐮴	𐮵	𐮶	𐮷	𐮸	𐮹	𐮺	𐮻	𐮼	𐮽	𐮾	𐮿	𐯀	𐯁	𐯂	𐯃	𐯄	𐯅	𐯆	𐯇	𐯈	𐯉	𐯊	𐯋	𐯌	𐯍	𐯎	𐯏	𐯐	𐯑	𐯒	𐯓	𐯔	𐯕	𐯖	𐯗	𐯘	𐯙	𐯚	𐯛	𐯜	𐯝	𐯞	𐯟	𐯠	𐯡	𐯢	𐯣	𐯤	𐯥	𐯦	𐯧	𐯨	𐯩	𐯪	𐯫	𐯬	𐯭	𐯮	𐯯	𐯰	𐯱	𐯲	𐯳	𐯴	𐯵	𐯶	𐯷	𐯸	𐯹	𐯺	𐯻	𐯼	𐯽	𐯾	𐯿	𐰀	𐰁	𐰂	𐰃	𐰄	𐰅	𐰆	𐰇	𐰈	𐰉	𐰊	𐰋	𐰌	𐰍	𐰎	𐰏	𐰐	𐰑	𐰒	𐰓	𐰔	𐰕	𐰖	𐰗	𐰘	𐰙	𐰚	𐰛	𐰜	𐰝	𐰞	𐰟	𐰠	𐰡	𐰢	𐰣	𐰤	𐰥	𐰦	𐰧	𐰨	𐰩	𐰪	𐰫	𐰬	𐰭	𐰮	𐰯	𐰰	𐰱	𐰲	𐰳	𐰴	𐰵	𐰶	𐰷	𐰸	𐰹	𐰺	𐰻	𐰼	𐰽	𐰾	𐰿	𐱀	𐱁	𐱂	𐱃	𐱄	𐱅	𐱆	𐱇	𐱈	𐱉	𐱊	𐱋	𐱌	𐱍	𐱎	𐱏	𐱐	𐱑	𐱒	𐱓	𐱔	𐱕	𐱖	𐱗	𐱘	𐱙	𐱚	𐱛	𐱜	𐱝	𐱞	𐱟	𐱠	𐱡	𐱢	𐱣	𐱤	𐱥	𐱦	𐱧	𐱨	𐱩	𐱪	𐱫	𐱬	𐱭	𐱮	𐱯	𐱰	𐱱	𐱲	𐱳	𐱴	𐱵	𐱶	𐱷	𐱸	𐱹	𐱺	𐱻	𐱼	𐱽	𐱾	𐱿	𐲀	𐲁	𐲂	𐲃	𐲄	𐲅	𐲆	𐲇	𐲈	𐲉	𐲊	𐲋	𐲌	𐲍	𐲎	𐲏	𐲐	𐲑	𐲒	𐲓	𐲔	𐲕	𐲖	𐲗	𐲘	𐲙	𐲚	𐲛	𐲜	𐲝	𐲞	𐲟	𐲠	𐲡	𐲢	𐲣	𐲤	𐲥	𐲦	𐲧	𐲨	𐲩	𐲪	𐲫	𐲬	𐲭	𐲮	𐲯	𐲰	𐲱	𐲲	𐲳	𐲴	𐲵	𐲶	𐲷	𐲸	𐲹	𐲺	𐲻	𐲼	𐲽	𐲾	𐲿	𐳀	𐳁	𐳂	𐳃	𐳄	𐳅	𐳆	𐳇	𐳈	𐳉	𐳊	𐳋	𐳌	𐳍	𐳎	𐳏	𐳐	𐳑	𐳒	𐳓	𐳔	𐳕	𐳖	𐳗	𐳘	𐳙	𐳚	𐳛	𐳜	𐳝	𐳞	𐳟	𐳠	𐳡	𐳢	𐳣	𐳤	𐳥	𐳦	𐳧	𐳨	𐳩	𐳪	𐳫	𐳬	𐳭	𐳮	𐳯	𐳰	𐳱	𐳲	𐳳	𐳴	𐳵	𐳶	𐳷	𐳸	𐳹	𐳺	𐳻	𐳼	𐳽	𐳾	𐳿	𐴀	𐴁	𐴂	𐴃	𐴄	𐴅	𐴆	𐴇	𐴈	𐴉	𐴊	𐴋	𐴌	𐴍	𐴎	𐴏	𐴐	𐴑	𐴒	𐴓	𐴔	𐴕	𐴖	𐴗	𐴘	𐴙	𐴚	𐴛	𐴜	𐴝	𐴞	𐴟	𐴠	𐴡	𐴢	𐴣	𐴤	𐴥	𐴦	𐴧	𐴨	𐴩	𐴪	𐴫	𐴬	𐴭	𐴮	𐴯	𐴰	𐴱	𐴲	𐴳	𐴴	𐴵	𐴶	𐴷	𐴸	𐴹	𐴺	𐴻	𐴼	𐴽	𐴾	𐴿	𐵀	𐵁	𐵂	𐵃	𐵄	𐵅	𐵆	𐵇	𐵈	𐵉	𐵊	𐵋	𐵌	𐵍	𐵎	𐵏	𐵐	𐵑	𐵒	𐵓	𐵔	𐵕	𐵖	𐵗	𐵘	𐵙	𐵚	𐵛	𐵜	𐵝	𐵞	𐵟	𐵠	𐵡	𐵢	𐵣	𐵤	𐵥	𐵦	𐵧	𐵨	𐵩	𐵪	𐵫	𐵬	𐵭	𐵮	𐵯	𐵰	𐵱	𐵲	𐵳	𐵴	𐵵	𐵶	𐵷	𐵸	𐵹	𐵺	𐵻	𐵼	𐵽	𐵾	𐵿	𐶀	𐶁	𐶂	𐶃	𐶄	𐶅	𐶆	𐶇	𐶈	𐶉	𐶊	𐶋	𐶌	𐶍	𐶎	𐶏	𐶐	𐶑	𐶒	𐶓	𐶔	𐶕	𐶖	𐶗	𐶘	𐶙	𐶚	𐶛	𐶜	𐶝	𐶞	𐶟	𐶠	𐶡	𐶢	𐶣	𐶤	𐶥	𐶦	𐶧	𐶨	𐶩	𐶪	𐶫	𐶬	𐶭	𐶮	𐶯	𐶰	𐶱	𐶲	𐶳	𐶴	𐶵	𐶶	𐶷	𐶸	𐶹	𐶺	𐶻	𐶼	𐶽	𐶾	𐶿	𐷀	𐷁	𐷂	𐷃	𐷄	𐷅	𐷆	𐷇	𐷈	𐷉	𐷊	𐷋	𐷌	𐷍	𐷎	𐷏	𐷐	𐷑	𐷒	𐷓	𐷔	𐷕	𐷖	𐷗	𐷘	𐷙	𐷚	𐷛	𐷜	𐷝	𐷞	𐷟	𐷠	𐷡	𐷢	𐷣	𐷤	𐷥	𐷦	𐷧	𐷨	𐷩	𐷪	𐷫	𐷬	𐷭	𐷮	𐷯	𐷰	𐷱	𐷲	𐷳	𐷴	𐷵	𐷶	𐷷	𐷸	𐷹	𐷺	𐷻	𐷼	𐷽	𐷾	𐷿	𐸀	𐸁	𐸂	𐸃	𐸄	𐸅	𐸆	𐸇	𐸈	𐸉	𐸊	𐸋	𐸌	𐸍	𐸎	𐸏	𐸐	𐸑	𐸒	𐸓	𐸔	𐸕	𐸖	𐸗	𐸘	𐸙	𐸚	𐸛	𐸜	𐸝	𐸞	𐸟	𐸠	𐸡	𐸢	𐸣	𐸤	𐸥	𐸦	𐸧	𐸨	𐸩	𐸪	𐸫	𐸬	𐸭	𐸮	𐸯	𐸰	𐸱	𐸲	𐸳	𐸴	𐸵	𐸶	𐸷	𐸸	𐸹	𐸺	𐸻	𐸼	𐸽	𐸾	𐸿	𐹀	𐹁	𐹂	𐹃	𐹄	𐹅	𐹆	𐹇	𐹈	𐹉	𐹊	𐹋	𐹌	𐹍	𐹎	𐹏	𐹐	𐹑	𐹒	𐹓	𐹔	𐹕	𐹖	𐹗	𐹘	𐹙	𐹚	𐹛	𐹜	𐹝	𐹞	
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--

A	• —	U	• • —
B	— • • •	V	• • • —
C	— • — •	W	• — • —
D	— • •	X	— • • —
E	•	Y	— • — •
F	• • — •	Z	— — • •
G	— — •		
H	• • • •		
I	• •		
J	• — — —		
K	— • — —		
L	• — • •		
M	— — —		
N	— •		
O	— — — —		
P	• — — •		
Q	— — — —		
R	• — • •		
S	• • •		
T	—		

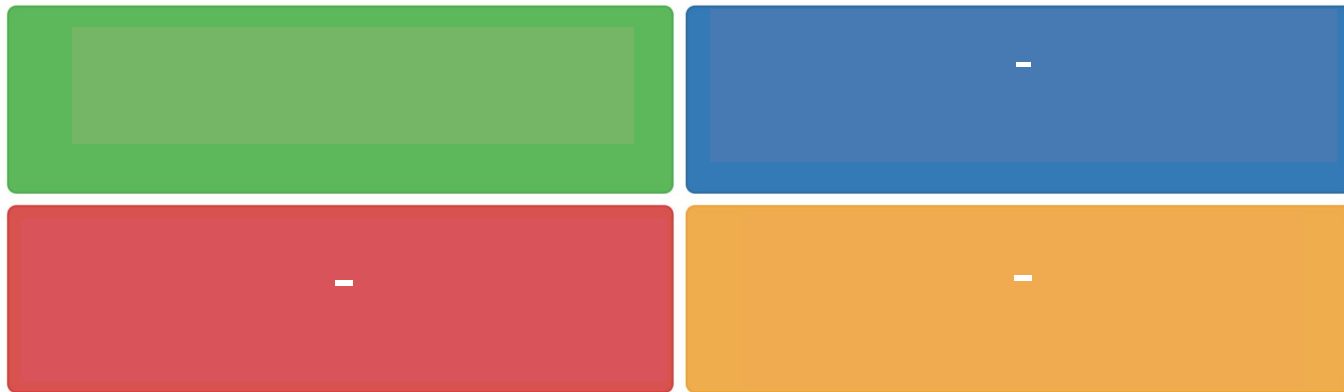
Morse Code



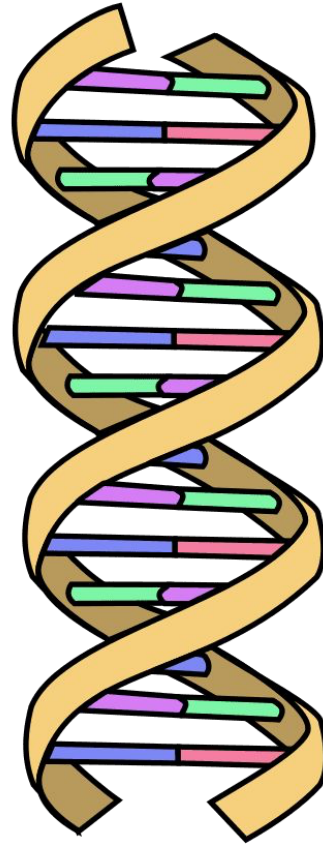
Cross Stitch

Participation Question

How do you count from 1 to 10
on your hands?



Data Representation in nature!



 = Adenine

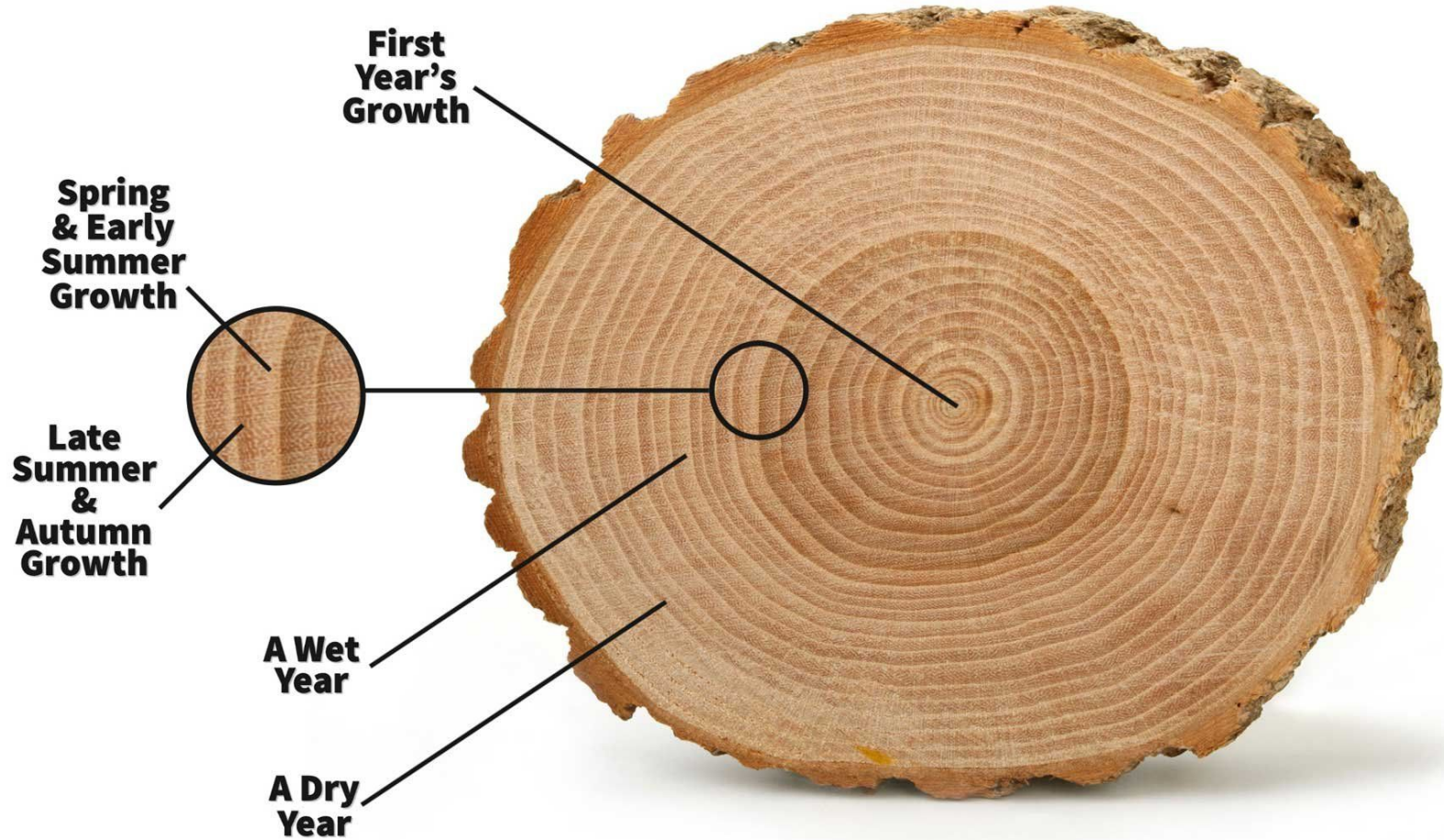
 = Thymine

 = Cytosine

 = Guanine

 = Phosphate
backbone

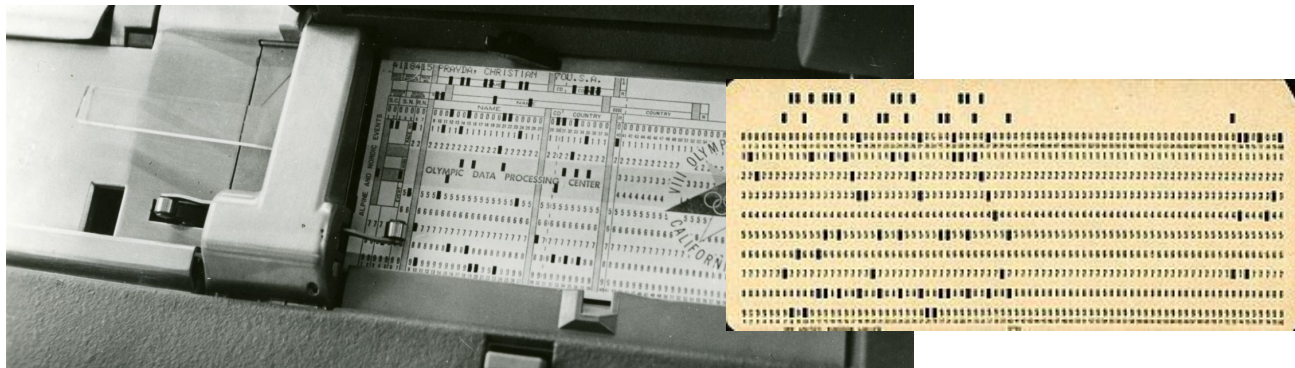
DNA



Data Representation with computers

Digital Data with Computers

- Computing technologies represent numbers, text, images, video, and sound using just two symbols: **0's and 1's**
- 0's and 1's are embodied as "high" (on) or "low" (off) signals on various electronic and optical storage media (e.g., punch cards, vacuum tubes, transistors, DVDs, etc.)



high	low
on	off
true	false
yes	no
+	-
1	0

Why is data represented in binary for computers?



transistors

Numbers in a Digital Era

- [Base-10] Decimal Numbers
 - E.g., 1, 2, 3, etc.
- [Base-2] Binary Numbers
 - E.g., 11001, 1001, 1111, etc.
- [Base-3] Ternary Numbers
 - E.g., 12_3 , 110_3 , 20_3 , etc.
- [Base-16] Hexadecimal Numbers
 - E.g., FF5733, AB4151, etc.



Decimal Numbers

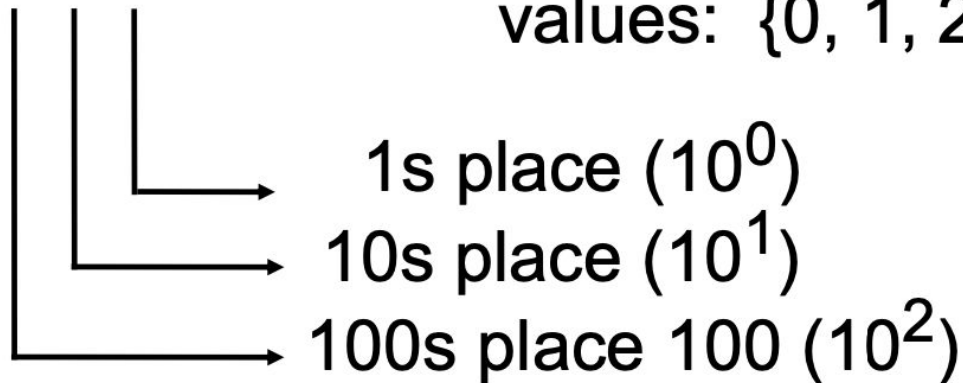
Decimal Numbers

Decimal numbers are base 10.

("decem" is Latin for 10).

In **decimal**, every column is worth 10 times as much as the previous one

2 4 8 ← Each digit can be any one of these 10 values: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}





Counting in Decimal

When counting in **decimal**:

- Add one to the right most digit
- If that digit has the highest value (**9**), then add one to the digit in the next column to the left and reset the column you're on to zero.
- If you run out of digits in the column to the left, you repeat the process

•	0	
•	1	...
•	2	
•	3	99
•	4	100
•	5	
•	6	
•	7	
•	8	
•	9	
		...
		10999
		11000



Counting in Decimal Example 1

$$\begin{array}{|c|c|c|} \hline 1 & 0 & 0 \\ \hline \end{array} + 1 \rightarrow \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline \end{array}$$

Add 1 to the
rightmost digit



Counting in Decimal Example 2

1 0 9 +1 → 1 1 0

Add 1 to the rightmost digit. 9 is the highest representation for a number, so we have to change 9 to 0 and increment the number to its left by 1



Binary Numbers



Bits

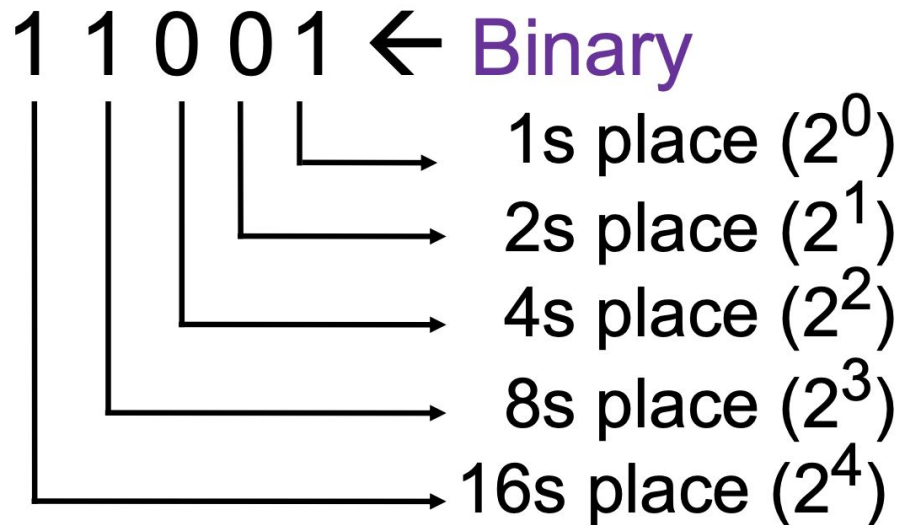
- A **bit** is short for a **binary digit**.
 - It is one 0 or 1. E.g., one "high" segment on a DVD or an "on" transistor.
- When a computer sees 1000000 (e.g. one "on" part followed by six "off" parts), it reads this as **binary** and *processes* this as **decimal 64**.
 - 64 and 1000000 are two representations of the same number
- When reading out binary numbers, **we say the digits in turn**
 - For "010" we say "zero one zero" or just "one zero" (We don't say "ten").
 - Sometimes we write this as **0b010** to make it clear it is a **binary** number.
 - You may also see a subscript 2 after the number (e.g., 010_2).

Binary Numbers

Binary numbers are base 2.

("binarius" is Latin for having two parts).

In binary, every column is worth 2 times as much as the previous one





Convert Binary to Decimal

0b 1 1 0 0 1

					1s place multiply by 1	(2^0):	1
				→	2s place multiply by 2	(2^1):	0
			→	4s place multiply by 4	(2^2):	0	
		→	8s place multiply by 8	(2^3):	8		
→	16s place multiply by 16	(2^4):	+	<u>16</u>			

Total in decimal (add them up): 25



Algorithm to Convert Decimal to Binary

- Start with a decimal number n (0 to 255)
 - Since 255 is the maximum, we need 8 bits.
- Check the highest power of 2 that fits into n :
 - If $n < 128$, set the 1st (leftmost) bit to 0; otherwise, set it to 1 and subtract 128 from n ($n=n-128$)
 - If $n < 64$, set the 2nd bit to 0; otherwise, set it to 1 and subtract 64 from n . ($n=n-64$)
 - If $n < 32$, set the 3rd bit to 0; otherwise, set it to 1 and subtract 32 from n . ($n=n-32$)
 - Continue this process for 16, 8, 4, 2, and finally 1.
- By the end, n will be either 0 or 1, which is the last (8th) bit.



Algorithm to Convert 217 to Binary

- $n = 217$
- Check the highest power of 2 that fits into n :
 - $\times 217 < 128 (2^7)$, 1st Bit = **1**
 - $n = 217 - 128 \Rightarrow 89$
 - $\times 89 < 64 (2^6)$, 2nd bit = **1**
 - $n = 89 - 64 \Rightarrow 25$
 - $\checkmark 25 < 32 (2^5)$, 3rd bit = **0**
 - *No subtraction required*
 - $\times 25 < 16 (2^4)$, 4nd bit = **1**
 - $n = 25 - 16 \Rightarrow 9$
 - $\times 9 < 8 (2^3)$, 5th Bit = **1**
 - $n = 9 - 8 \Rightarrow 1$
 - $\checkmark 1 < 4 (2^2)$, 6th bit = **0**
 - *No subtraction required*
 - $\checkmark 1 < 2 (2^1)$, 7th bit = **0**
 - *No subtraction required*
 - **Final bit = $n = 1 (2^0) \rightarrow$ 8th bit = **1****
- **Final Answer: 11011001**



Binary Numbers

- Binary numbers only use two digits: 0 and 1, and their place values are based on powers of 2.

Number 9 in Binary:

256	128	64	32	16	8	4	2	1
0	0	0	0	0	1	0	0	1

9 = 000001001



Counting in Binary

When counting in **binary**:

- Add one to the right most digit
- If that digit has the highest value (**1**), then add one to the digit in the next column to the left and reset the column you're on to zero.
- If you run out of digits in the column to the left, you repeat the process

- 0
- 1
- 10
- 11
- 100
- 101
- ...
- 101101
- 101110
- 110001



Counting in Binary Example 1

1 0 0 +1 → 1 0 1

Add 1 to the
rightmost digit



Clicker: Addition in Binary



iClicker

$$\begin{array}{ccccccc} 1 & 0 & 1 & +1 & \rightarrow & ? & ? & ? \\ \hline \end{array}$$

What number should go on the right side?
(note: both #s are in binary)

- A) 100
- B) 101
- C) 110
- D) 111



Clicker: Addition in Binary



iClicker

$$\begin{array}{ccc} 1 & 0 & 1 \\ \hline \end{array} + 1 \rightarrow \begin{array}{ccc} ? & ? & ? \\ \hline \end{array}$$

What number should go on the right side?
(note: both #s are in binary)

A) 100

B) 101

C) 110

D) 111

Q: The 8-bit binary representation of 57 is 00111001. What is the 8-bit binary representation of 58?



- A. 01011110
- B. 00111111
- C. 00111010
- D. 0011100010001

Q: The 8-bit binary representation of 57 is 00111001. What is the 8-bit binary representation of 58?



A. 01011110

B. 00111111

C. 00111010

D. 0011100010001



Wrap up